RESEARCH PAPER · SMART ENERGY SYSTEMS

AI-Based Smart Electricity Consumption Prediction and Anomaly Detection

A benchmark study and deployable web system on four years of household meter data — eight forecasting models, four anomaly detectors, and a one-click deployment of the best pipeline.

Mythos0
Independent Researcher

DATASET	UCI Individual Household Electric Power Consumption (2006–2010)
BEST MODEL	XGBoost · RMSE 0.588 kW · +22.4% skill over seasonal naive
ANOMALIES	Residual z-score · ROC-AUC 0.885 · spike–dropout asymmetry
ARTIFACTS	Code, models, figures, Kaggle notebook, Render deployment

November 2026

github.com/mythos0/ai-smart-energy-system

AI-Based Smart Electricity Consumption Prediction and Anomaly Detection:

A Benchmark Study and Deployable Web System on Four Years of Household Meter Data

Mythos0
Independent Researcher
November 2026

Abstract—This paper presents an end-to-end artificial intelligence system for short-term electricity consumption prediction and anomaly detection at the individual household level. We benchmark eight forecasting models—two seasonal-naïve baselines, Ridge regression, Holt–Winters exponential smoothing, Random Forest, XGBoost, LightGBM, and a direct multi-step long short-term memory (LSTM) network—on the UCI Individual Household Electric Power Consumption dataset (2.08 million minute-level readings, December 2006 to November 2010), under a strict day-ahead evaluation protocol in which every feature available to a model is at least 24 hours old. Gradient-boosted trees prove the strongest family: XGBoost attains a test RMSE of 0.588 kW, a 22.4% improvement over the seasonal-naïve baseline, with the LSTM competitive at 0.592 kW and the lowest MAE and MAPE. For anomaly detection we compare four approaches—forecast-residual z-scoring, Isolation Forest, One-Class SVM, and an LSTM autoencoder—on a controlled benchmark with 112 injected anomalies spanning spikes, level shifts, and dropouts. The residual method dominates (ROC-AUC 0.885, F1 0.482), and the per-type analysis exposes a sharp asymmetry: spikes are detected with 93–98% recall, whereas near-zero dropout events largely evade every tested detector. A case study on the unmodified test period surfaces plausible real anomalies, including multi-hour evening consumption excursions exceeding 4 kW. Finally, we package the selected models behind a Flask web service with an interactive dashboard and a REST API, deployable to the Render platform, closing the loop from data to an operational decision-support tool. All code, trained models, figures, and the training notebook are released publicly.

Keywords—load forecasting; anomaly detection; XGBoost; LSTM; autoencoder; smart meter; time series; deployed machine learning

1 Introduction

Electricity is consumed the instant it is produced, so every actor in the power value chain—from grid operators balancing frequency to retailers pricing tariffs and households sizing rooftop solar and batteries—benefits from knowing consumption in advance. The same household meter stream enables a second, complementary capability: detecting when consumption behaves abnormally. Abnormal patterns can indicate a failing appliance left running, an inefficient device drawing standby power, or simply an unexpected change in occupancy that invalidates existing schedules and contracts. Forecasting and anomaly detection are therefore two sides of the same smart-metering pipeline, and practical systems increasingly require both, delivered together and evaluated on equal footing.

Household-level forecasting is, however, substantially harder than its aggregate counterpart. System-level load is the convolution of thousands of consumers, and the law of large numbers smooths individual behaviour into a series dominated by calendar cycles and weather; predictive models routinely reach coefficients of determination above 0.9 at the system level [1, 2]. A single household's hourly consumption instead reflects the stochastic timing of a handful of appliance events—a kettle, a dryer, a water heater—superimposed on a slowly varying baseline. The predictable component shrinks, and the irreducible noise grows accordingly [3]. This raises a first question that this paper addresses systematically: *how much* of a household's next-day consumption is actually predictable, and which model family captures it best?

A second question concerns anomaly detection methodology. Published evaluations of smart-meter anomaly detectors differ widely in protocol, and several popular methods are examined only on data whose anomalies are of a single, easy kind. We argue that a fair comparison must (i) inject anomaly types of contrasting sign and duration—sharp high-consumption spikes, sustained level shifts, and prolonged near-zero dropouts—and (ii) contaminate the lagged features that detectors consume, rather than only the

evaluation target, so that the measured robustness reflects operational conditions. Under this protocol we quantify how four standard detectors, spanning three distinct paradigms, trade off against one another.

Third, machine-learning research on this problem often stops at metrics tables. To be useful, a trained model must be reachable by non-specialists and by other software. We therefore package the best-performing forecaster and the residual anomaly detector into a lightweight web application with an interactive dashboard and a documented REST API, deployable on a free-tier platform (Render), and we report the engineering decisions that keep the service inside a 512 MB memory budget.

The contributions of this work are:

1. A controlled benchmark of eight forecasting models on four years of real household meter data under a *strict* 24-hour-ahead protocol, with hyperparameters selected on a chronological validation split and all results reported on an untouched eight-month test period.
2. A contamination-aware anomaly detection study comparing residual z-scoring, Isolation Forest, One-Class SVM, and an LSTM autoencoder across three injected anomaly types, plus a case study of the strongest events found in the real, unmodified test data.
3. An analysis of where each approach fails: the asymmetry between easily detected high-consumption spikes and nearly invisible dropouts, and the evening-hour concentration of forecast errors.
4. A deployable web system (Flask, Chart.js, REST API, Render configuration) that serves day-ahead forecasts, live forecasts on user-supplied histories, anomaly scores, and the full benchmark leaderboard, together with the complete public repository of code, models, figures, and a reproducible Kaggle notebook.

2 Related Work

Load forecasting is one of the oldest applications of statistical time-series modelling. Alfares and Nazeeruddin [1] classify the pre-machine-learning literature into regression, exponential smoothing, and knowledge-based approaches; Hong and Fan [2] later survey the shift toward probabilistic and data-driven methods. Classical exponential smoothing with trend and seasonality originates with Winters [4] and remains a strong, assumption-light baseline that we include in our benchmark. Seasonal-trend decomposition by loess (STL) [5] provides the standard lens for inspecting the calendar structure of demand series.

Within machine learning, tree ensembles and neural networks dominate recent practice. Random Forests [6] aggregate decorrelated decision trees and handle mixed tabular features with little preprocessing. Gradient-boosted trees refine the idea by sequentially fitting shallow trees to residual errors; XGBoost [7] popularised regularised, histogram-based implementations that win a large share of tabular data competitions, and LightGBM [8] accelerated training further with leaf-wise growth and gradient-based one-side sampling. On the neural side, recurrent architectures were designed specifically to retain long-range temporal context; LSTM [9] alleviates the vanishing-gradient problem of vanilla recurrent networks and has become the reference deep architecture for short-term load forecasting. Kong et al. [3] apply LSTM networks to residential load at various horizons and find that, while deep models outperform classical baselines, individual-customer load remains far less predictable than aggregate load—an observation our results reproduce and quantify.

Anomaly detection methodology is surveyed by Chandola et al. [10], who classify approaches into statistical, proximity-based, and one-class boundary methods. Isolation Forest [11] detects outliers by random recursive partitioning, exploiting the fact that anomalies isolate in fewer splits; its linear complexity makes it a default choice for streaming features. The One-Class SVM [12] learns a boundary enclosing the bulk of a kernelised density and flags points outside it. Reconstruction-based methods train a model to rebuild normal patterns and treat reconstruction error as an anomaly score; Sakurada and Yairi [13] demonstrated that autoencoders capture nonlinear normality that linear PCA misses, establishing the recipe we adopt for our LSTM autoencoder. In the energy domain, forecast-residual methods are attractive because they reuse the forecaster already required by the system: the residual between actual and predicted consumption is standardised, and large deviations are flagged [14]. Our study compares all four families under a feature-contaminating injection protocol with sign-diverse anomaly types on this benchmark dataset, on equal footing with the forecasting task itself.

Table 1: Dataset summary and chronological splits (after cleaning).

Property	Value	Notes
Raw rows (minute level)	2,075,259	47 months, Dec 2006 – Nov 2010
Rows with missing values	25,979 (1.25%)	contiguous sensor outages
Rows after interpolation	2,050,108	linear-in-time, gaps $\leq$ 60 min
Hourly rows after aggregation	34,176	mean of global active power (kW)
Hourly mean / std / max	1.09 / 0.90 / 6.56 kW	
Daily energy (mean)	$\approx$ 26.1 kWh	
Train split	21,951 hours	2006-12-23 – 2009-06-30
Validation split	6,480 hours	2009-07-01 – 2010-03-31
Test split	5,554 hours	2010-04-01 – 2010-11-26

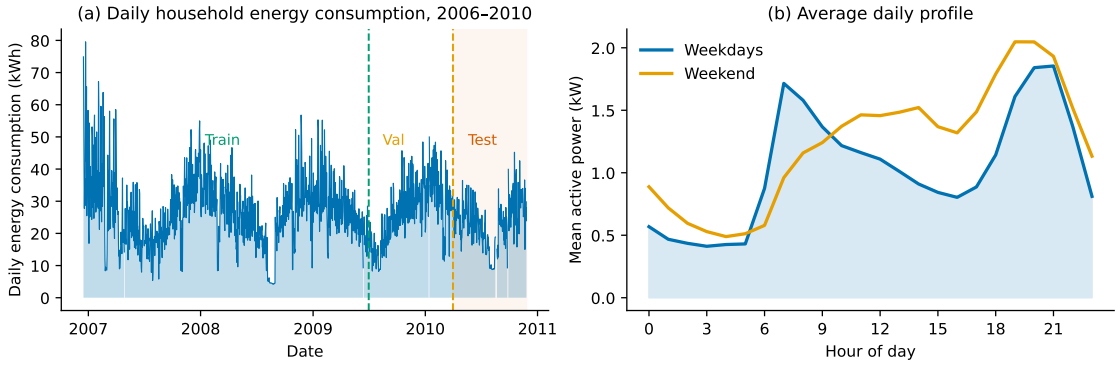


Figure 1: Dataset overview. (a) Daily energy consumption with chronological train/validation/test boundaries; (b) average daily profile, weekdays versus weekend, showing the morning and evening ramps that motivate hour-of-day features.

3 Dataset and Exploratory Analysis

3.1 Source data

We use the UCI Individual Household Electric Power Consumption dataset [15], which records one household in Sceaux, France (7 km south of Paris), at one-minute resolution for 47 months (December 2006 to November 2010). The archive contains 2,075,259 rows and nine attributes: global active and reactive power (kW), voltage (V), global intensity (A), and three appliance-group sub-meterings (kitchen; laundry room; water heater and air conditioner, in watt-hours). The household also owns unmetered loads, whose contribution is recoverable as the residual of the global measurement and the sub-meters. Table 1 summarises the dataset and our processed splits.

3.2 Preprocessing

Minute-level rows containing missing values (1.25% of the archive, occurring in contiguous outages) were interpolated linearly in time with a 60-minute cap, and hours with insufficient raw coverage were dropped. The cleaned series was aggregated to hourly means of global active power, yielding 34,176 hourly observations. We deliberately exclude instantaneous electrical covariates (voltage, intensity) from the feature set: in a genuine day-ahead setting their future values are unknown, and including them would leak contemporaneous information.

3.3 Exploratory findings

Figure 1(a) shows daily energy over the full four years with the chronological split boundaries. The series is stationary in the mean with no long-term drift, but exhibits dense bursts of high-consumption days in winter (heating-related loads) and occasional multi-day low-consumption troughs consistent with absences. Figure 1(b) contrasts average weekday and weekend profiles: a sharp weekday morning ramp peaking near 07:00–08:00, an evening peak around 18:00–20:00, and a flatter, delayed weekend morning. The mean hourly load is 1.09 kW (about 26.1 kWh per day).

Figure 2(a) maps mean power by month and hour of day, revealing strong daily periodicity in all months, elevated winter consumption, and the double-peak structure of the daily profile. Figure 2(b) applies STL decomposition [5] to a two-month window: the trend absorbs slow weather-driven variation while the daily

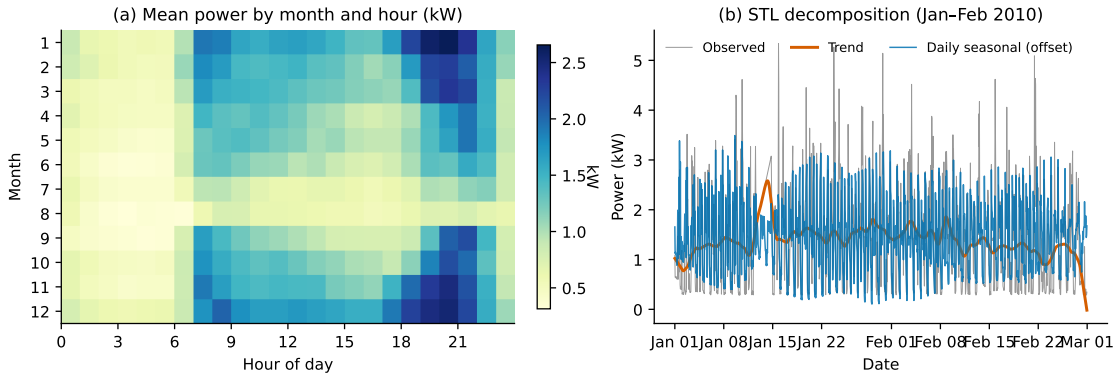


Figure 2: Seasonality analysis. (a) Mean active power by month and hour of day; (b) STL decomposition (period 24 h, robust) of January–February 2010 showing observed, trend, and daily seasonal components.

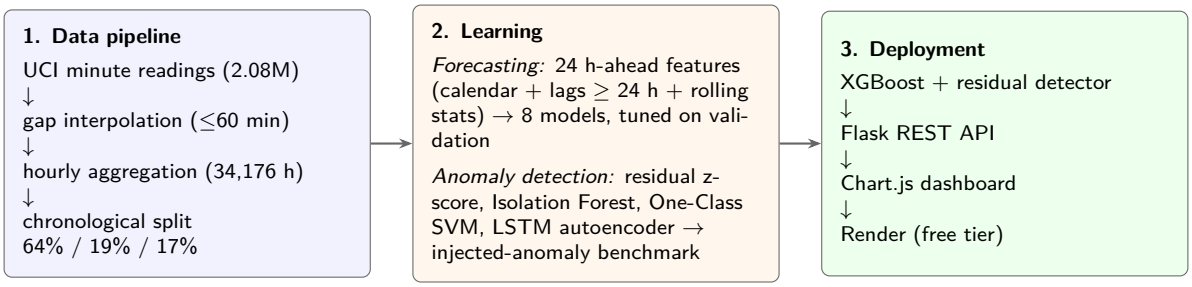


Figure 3: System architecture: data pipeline, learning stage, and deployment.

seasonal component captures the recurring profile. These structures justify the lag and calendar features used by the supervised models, and the residual component foreshadows the difficulty of the day-ahead task: hour-to-hour movements of one to two kilowatts are routine.

4 Methodology

Figure 3 summarises the processing pipeline from raw data to the deployed service. The remainder of this section details each stage.

4.1 Problem formulation

Let y_t denote the mean active power (kW) during hour t. The task is day-ahead forecasting: at the end of day $d-1$, predict all 24 hourly values of day d. Formally, for every target hour t we seek an estimate

$$\hat{y}_t = f(\mathcal{F}_{<t-23\mathrm{h}}), \tag{1}$$

where the feature set $\mathcal{F}$ is restricted to information at least 24 hours old. This is stricter than the common one-step-ahead protocol, which feeds the most recent hour to the model, and it matches the operational day-ahead setting in which schedules must be committed in advance.

4.2 Feature engineering

For each target hour t we construct 11 features, all computed from data older than $t-23$ hours:

- *Calendar:* hour of day, day of week, month, weekend indicator, day of year.
- *Autoregressive lags:* y_{t-24}, y_{t-48}, y_{t-168} (same hour yesterday, two days ago, and one week ago).
- *Rolling statistics:* the mean and standard deviation of the 24 hours ending at y_{t-24}, and the mean of the 168 hours ending at y_{t-24}.

The lag-24 and lag-168 features encode the daily and weekly periodicity visible in Figure 2; the rolling statistics provide a local baseline level that adapts to slow regime changes such as seasonal drift or occupancy shifts.

Table 2: Model configurations after validation-based selection.

Model	Selected configuration	Final fit time
Seasonal naïve	none	–
Ridge	$\alpha = 0.1$ (cyclical encoding)	< 1 s
Holt–Winters	additive seasonal (24), damped trend, 8-week rolling refits	171 s
Random Forest	500 trees, max depth 18, min leaf 1	53 s
XGBoost	800 trees, depth 4, learning rate 0.03	0.7 s
LightGBM	800 trees, 31 leaves, learning rate 0.03	1.7 s
LSTM	2 layers, hidden 64, horizon 24, early stop (epoch 6 of 40)	336 s

4.3 Forecasting models

Eight models of increasing capacity are compared; Table 2 lists the configuration selected for each on the validation split.

1. **Seasonal naïve (24 h and 168 h):** $\hat{y}_t = y_{t-24}$ and $\hat{y}_t = y_{t-168}$. These require no training and define the skill baseline.
2. **Ridge regression:** linear model on cyclical (sine/cosine) encodings of hour, day of week, and month, plus the lag features; the ℓ_2 penalty is selected on validation RMSE.
3. **Holt–Winters exponential smoothing:** additive trend with damping and daily seasonality (period 24), refit daily on a trailing eight-week window so that each day-ahead forecast uses only past data; additive and multiplicative seasonal variants are compared on validation.
4. **Random Forest [6]:** 500 trees; depth and minimum leaf size selected on validation.
5. **XGBoost [7]:** gradient-boosted trees with the histogram method, subsampling of rows and columns, and ℓ_2 regularisation.
6. **LightGBM [8]:** leaf-wise growth with the same regularisation family.
7. **LSTM network [9]:** a direct multi-step architecture that maps the past 168 hours (standardised consumption plus cyclical calendar channels) through a two-layer LSTM (hidden size 64) to all 24 target hours at once, trained with Smooth-L_1 loss and early stopping on validation loss. For the day-ahead score reported in Table 3, hour t is scored with the 24th output of the window ending 24 hours earlier—exactly the feature timeliness of Eq. (1).

4.4 Anomaly detection framework

We compare four detectors drawn from the statistical, density, boundary, and reconstruction paradigms of unsupervised anomaly detection:

1. **Residual z-score:** with the best forecaster f, the residual $r_t = y_t - \hat{y}_t$ is standardised by the residual standard deviation σ estimated on the clean validation split,

$$z_t = \frac{|y_t - \hat{y}_t|}{\sigma}, \qquad \text{anomalous if } z_t > 3. \tag{2}$$

2. **Isolation Forest [11]:** 300 trees on standardised consumption features (current value, lags, rolling statistics, calendar).
3. **One-Class SVM [12]:** RBF kernel, ν matched to the expected contamination.
4. **LSTM autoencoder [13]:** a 24-hour window is encoded to a 32-unit vector and decoded back to 24 values; the per-hour anomaly score is the mean squared reconstruction error over all windows containing that hour, which localises scores better than the conventional window-end attribution.

4.5 Evaluation protocol

Forecast quality is measured by RMSE, MAE, MAPE, and R^2 on the untouched test split, together with the forecast skill score

$$\text{Skill} = 1 - \frac{\text{RMSE}}{\text{RMSE}_{\text{naive-24h}}}, \tag{3}$$

the fractional RMSE improvement over the daily seasonal naïve baseline.

Table 3: Day-ahead (24 h) forecasting results on the test period (Apr–Nov 2010, 5,554 hours). Best value per column in bold.

Model	RMSE (kW)	MAE (kW)	MAPE (%)	R^2	Skill (%)
XGBoost	**0.5875**	0.4328	64.91	**0.318**	**22.4**
LSTM	0.5920	**0.4213**	**55.86**	0.307	21.8
LightGBM	0.5931	0.4354	65.93	0.304	21.6
Random Forest	0.5937	0.4400	68.09	0.303	21.5
Ridge	0.6060	0.4513	65.36	0.274	19.9
Seasonal naïve (24 h)	0.7568	0.5101	65.97	−0.132	0.0
Holt–Winters	0.7860	0.5949	92.40	−0.222	−3.9
Seasonal naïve (168 h)	0.7948	0.5500	75.36	−0.249	−5.0

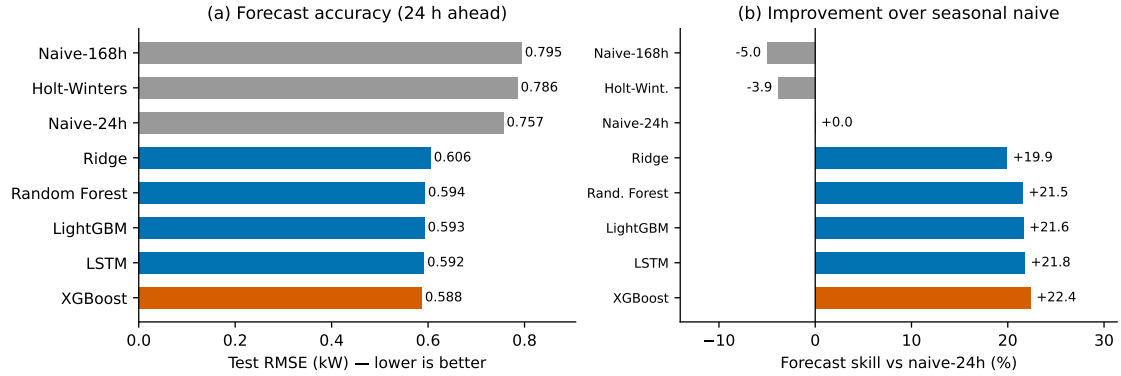


Figure 4: Forecast accuracy on the test period. (a) Test RMSE by model; (b) forecast skill (Eq. (3)) relative to the seasonal-naïve 24 h baseline. Red highlights the selected deployment model.

For anomaly detection, the dataset provides no ground-truth anomaly labels, so we inject controlled anomalies into the *test region only*: 112 anomalous hours (2.02% of test hours) comprising 44 single-hour spikes (consumption multiplied by 2.5–4.0 plus 1.5 kW, capped at 8 kW), 33 hours of level shifts (2–6 hour blocks raised by 60–150%), and 35 hours of dropouts (4–12 hour blocks forced to 0.02–0.10 kW). Lag features are then *recomputed from the contaminated series*, so detectors must cope with polluted context exactly as they would in operation. Detection quality is reported as precision, recall, and F1 at the operating point matching the true prevalence (top-2% of scores), plus threshold-free ROC-AUC and PR-AUC, and recall is decomposed by anomaly type. A case study on the clean, unmodified test period complements the synthetic benchmark.

5 Results

5.1 Forecasting

Table 3 reports test-period performance for the strict day-ahead task, and Figure 4 visualises the ranking. Three families of observations emerge.

First, **tree ensembles lead the ranking**. XGBoost achieves the best RMSE (0.5875 kW) and R^2 (0.318), a 22.4% skill improvement over the seasonal naïve—remarkable given that the baseline already exploits the strong daily periodicity. LightGBM and Random Forest follow within 1% of the leader, confirming that the gradient-boosting advantage on tabular problems carries over to day-ahead household load. The LSTM is statistically competitive on RMSE (0.5920 kW) and actually *wins* on MAE (0.4213 kW) and MAPE (55.9%): the network errs less on typical hours, while trees occasionally produce larger misses that RMSE penalises quadratically. From a deployment standpoint the models are interchangeable in accuracy; XGBoost is selected for its 0.7 s final fit time, small artefact size (0.8 MB), and CPU-only inference.

Second, **the achievable R^2 is modest by aggregate-load standards**. A value near 0.32 (versus >0.9 typical for system load) quantifies how much of a single household's next-day hourly consumption is governed by unpredictable occupant behaviour rather than calendar structure [3]. The MAPE values, likewise high at 56–68%, reflect the many near-zero overnight hours in which relative errors blow up; MAE in kilowatts is the more interpretable scale here.

Third, **both naïve persistence and Holt–Winters underperform the mean**. Holt–Winters trails even the 24-hour naïve baseline: the damped-trend extrapolation of a rolling window adds noise on a series whose level wanders with occupancy, and its 92% MAPE illustrates how poorly an unregularised seasonal

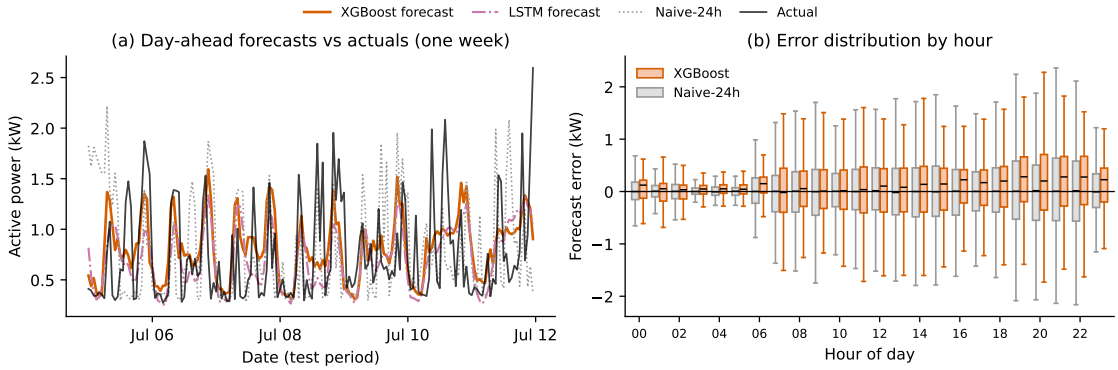


Figure 5: Forecast behaviour on the test period. (a) One week of actual consumption with day-ahead forecasts from XGBoost, LSTM, and the seasonal naïve; (b) distribution of forecast errors by hour of day (XGBoost versus naïve-24 h), showing the widest spread and largest positive bias during evening hours.

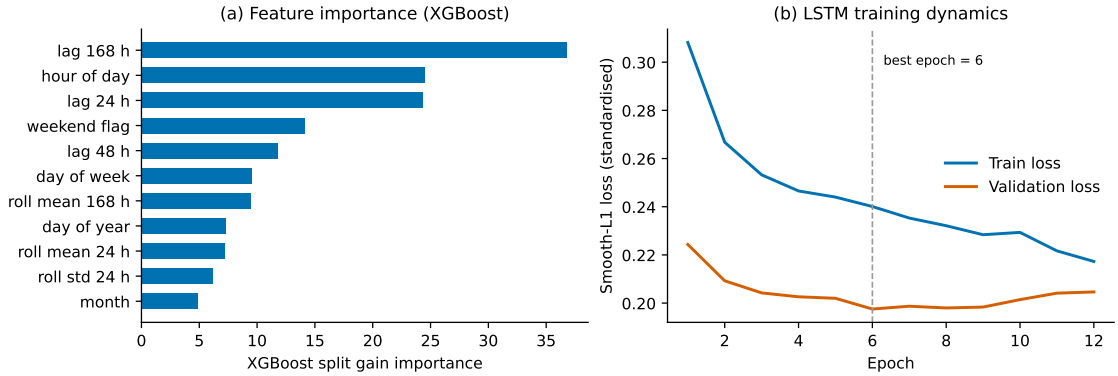


Figure 6: Model diagnostics. (a) XGBoost split-gain feature importance: the previous day's same-hour value and weekly structure dominate, with rolling means providing local level; (b) LSTM training and validation loss (standardised Smooth-L_1) with early stopping at the best validation epoch.

mean tracks low-consumption hours. This is a useful caution against reaching for classical smoothing as a default at the household level.

Figure 5(a) illustrates a representative week: all models track the daily shape, but evening peaks—where consumption is largest and most behaviour-driven—attract the biggest errors. Panel (b) quantifies this: error spread is widest between 17:00 and 22:00, where a drying cycle or a dinner with guests can double the load relative to the previous day at the same hour. The naïve baseline shows a systematic positive evening bias that the learned models largely remove.

Figure 6(a) ranks XGBoost features by split gain. The 24-hour lag is the single most used split, followed by the 168-hour lag and the 24-hour rolling mean—a compact confirmation that the model's predictive power comes from daily and weekly persistence around an adaptive local level, with calendar features supplying the remaining structure. Panel (b) shows the LSTM converged within six epochs and began overfitting shortly after; early stopping on the validation split was essential.

5.2 Anomaly detection

Table 4 reports the injected-anomaly benchmark and Figure 7 visualises the trade-offs.

The **residual z-score is the strongest detector overall** (ROC-AUC 0.885, F1 0.482), and it is the only method with non-zero recall on every anomaly type. Its advantage is structural: the XGBoost forecaster contextualises each hour against calendar and persistence features, so the residual isolates deviations from *expected* consumption rather than from the global feature distribution. A fixed 3-sigma threshold performs nearly identically (F1 0.469), so the operating point is not sensitive in this regime.

The **per-type decomposition is the most instructive result**. Spikes—sudden single-hour excesses—are detected almost perfectly by the two learned models (93–98% recall) and by half of the SVM decisions. Level shifts are detected only by the residual method (33%), which notices the sustained gap between actual and predicted level, while window-based methods average the anomaly away. Dropouts (near-zero blocks mimicking vacancy or disconnection) escape *all* detectors (0–6% recall): a 0.05 kW reading is statistically indistinguishable from a normal night hour when the local baseline is only 0.3–0.8 kW, and the

Table 4: Anomaly detection on the injection benchmark (112 anomalous hours, 2.02% of test; operating point set to true prevalence). Best value per column in bold.

						Recall by type		
Detector	**Prec.**	**Recall**	**F1**	**ROC-AUC**	**PR-AUC**	**Spike**	**Shift**	**Drop.**
Residual z-score (XGB)	**0.482**	**0.482**	**0.482**	**0.885**	**0.488**	**0.932**	**0.333**	**0.057**
LSTM-AE	0.402	0.402	0.402	0.721	0.314	0.977	0.061	0.000
One-Class SVM	0.232	0.232	0.232	0.726	0.130	0.523	0.091	0.000
Isolation Forest	0.196	0.196	0.196	0.731	0.114	0.409	0.121	0.000

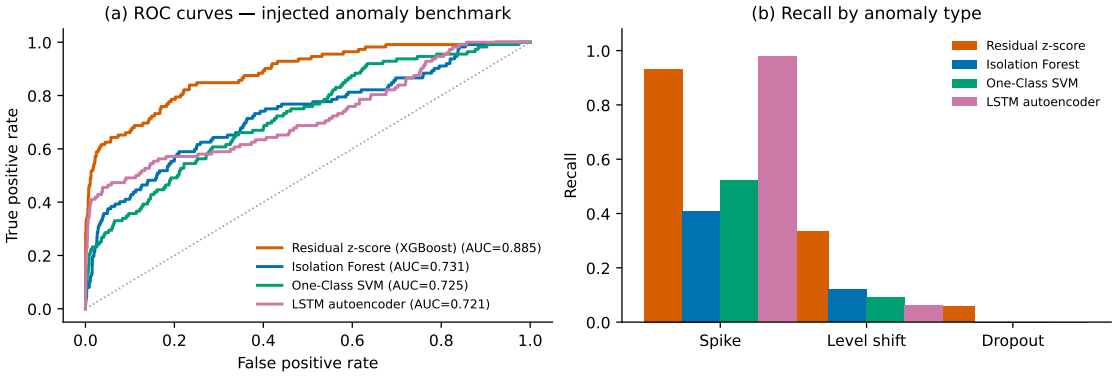


Figure 7: Anomaly detection benchmark. (a) ROC curves of the four detectors on the injection test; (b) recall decomposed by anomaly type, exposing the spike–dropout asymmetry: spikes are near-certainly detected, dropouts almost never.

absolute residual falls well inside three validation sigmas. Detecting low-side anomalies therefore requires relative or asymmetry-aware scores, which we identify as the clearest direction for future work.

The unsupervised feature-space methods (Isolation Forest, One-Class SVM) cluster in the 0.72–0.73 ROC-AUC band: their scores respond to feature-space isolation, which the contaminated lag features partially camouflage. The LSTM autoencoder reaches 0.402 F1 thanks to excellent spike recall but contributes nothing on sustained anomalies under our point-wise localisation.

5.3 Case study: real anomalies

Applied to the clean test period, the residual detector flags 55 hours (0.99%) beyond three sigmas. Table 5 lists the strongest merged events; Figure 8(a) shows one week of detections and panel (b) the detector operating against injected anomalies with its score trace.

The events are behaviourally plausible: single late-evening surges of 5–6 kW (consistent with high-wattage appliances or guests), and one remarkable *morning* excursion on 4 November 2010 in which consumption ran above 4 kW for four hours against a forecast of 1.2 kW. All top events are high-side deviations, mirroring the benchmark finding that the absolute-residual detector is asymmetrically sensitive to excess consumption.

6 Deployed Web System

To close the gap between benchmark and practice, the selected XGBoost forecaster and the residual anomaly detector are packaged as a Flask service (Figure 9). The dashboard offers four views: a *forecast* tab that backtests any day of the test period—showing the preceding week of actual consumption, the day-ahead forecast, and a $\pm 1.96\sigma$ prediction band—and accepts user-supplied hourly histories for genuine live forecasts; an *anomalies* tab that plots consumption, z-scores, and flagged hours for any week, alongside the strongest real events; a *models* tab rendering both leaderboards; and an *API* tab documenting the REST interface. The API exposes `GET /api/forecast?date=`, `POST /api/forecast` for live inference (minimum 216 hourly values; the model recursively consumes its own predictions beyond the first horizon), `GET /api/anomaly?start=&end=`, `GET /api/history`, `GET /api/models`, and `GET /api/anomaly/casestudy`, all returning JSON.

Engineering choices target the Render free tier (512 MB RAM): the service loads only the 0.8 MB XGBoost artefact plus pre-aggregated hourly data (3.9 MB) and residual scores, uses gunicorn with a single worker and four threads, and deliberately excludes deep-learning runtimes, which would inflate the image

Table 5: Strongest anomaly events detected on the unmodified test period (consecutive flagged hours merged; events within two hours joined).

Start	End	Hours	Max $\lvert z \rvert$	Actual (kW)	Forecast (kW)
2010-11-20 18:00	2010-11-20 18:00	1	5.2	5.63	2.31
2010-11-04 08:00	2010-11-04 11:00	4	5.1	4.42	1.17
2010-11-04 19:00	2010-11-04 20:00	2	4.7	4.46	1.49
2010-07-01 00:00	2010-07-01 00:00	1	4.5	3.33	0.45
2010-10-18 09:00	2010-10-18 19:00	5	4.5	3.84	0.99
2010-05-15 23:00	2010-05-15 23:00	1	4.0	3.82	1.32

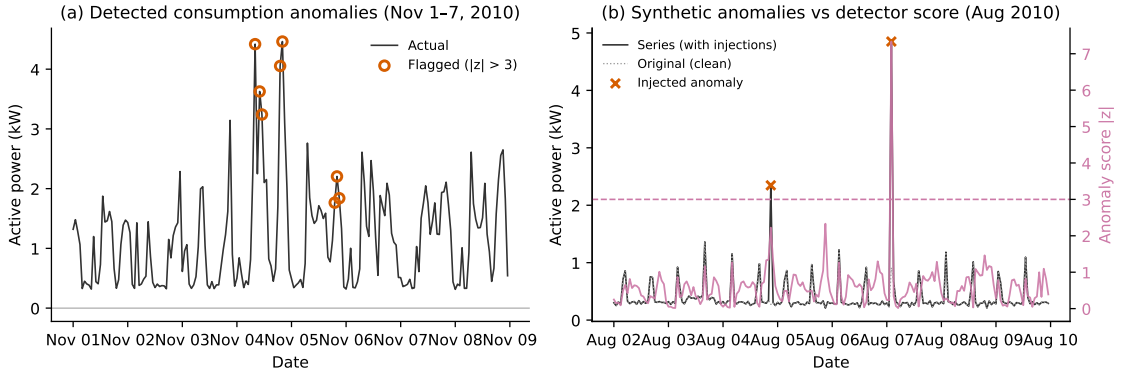


Figure 8: (a) Detected consumption anomalies ($|z| > 3$, circled) during a clean test week; (b) the detector score (right axis, purple) tracking injected anomalies (crosses) in a synthetic window, with the 3-sigma decision threshold dashed.

by two orders of magnitude for marginal accuracy gain. Every request after cold start is served from memory; a live 24-hour forecast adds only recursive feature construction and 24 tree traversals. A `render.yaml` blueprint, pinned requirements, and a health check (`/api/health`) make deployment a one-click operation from the public repository.

7 Discussion and Limitations

Predictability ceiling. The clustering of all learned models within a 1% RMSE band, far above the naïve baselines but far below aggregate-load accuracy, suggests the remaining error is dominated by irreducible occupant stochasticity rather than model capacity. In this regime, model choice should be driven by operational criteria—fit time, artefact size, inference latency—which favours gradient-boosted trees for embedded and edge deployment.

Asymmetric detectability. The near-perfect spike recall versus near-zero dropout recall is the sharpest finding of the anomaly study. It implies that systems built on absolute forecast residuals will reliably catch excessive consumption (often the financially significant failures: heaters left on, faulty appliances) but will miss vacancy and disconnection events. Relative or prediction-conditional scoring, and detectors that model the lower tail of the consumption distribution explicitly, are promising remedies.

Single household, synthetic anomalies. All results derive from one meter in one climate, and the injected anomalies are statistically simpler than real faults: injection is a necessity on an unlabelled dataset. The methodology transfers directly—features, protocol, and deployment are household-agnostic—but absolute accuracies will differ with appliance mix, occupancy patterns, and climate, and labelled field data would be required for definitive claims.

8 Conclusion

We presented a complete, reproducible study of AI-based consumption prediction and anomaly detection for a single household, spanning data engineering, an eight-model day-ahead forecasting benchmark, a contamination-aware four-detector anomaly evaluation, error and failure analysis, and a deployed web system. Gradient boosting (XGBoost) delivered the best test RMSE (0.588 kW, 22.4% skill over the seasonal naïve), with an LSTM competitive and best on MAE/MAPE; forecast-residual z-scoring dominated anomaly detection (ROC-AUC 0.885), and the per-type analysis revealed a fundamental spike–dropout asymmetry that defines the open problem for low-side anomaly detection. The accompanying repository releases all

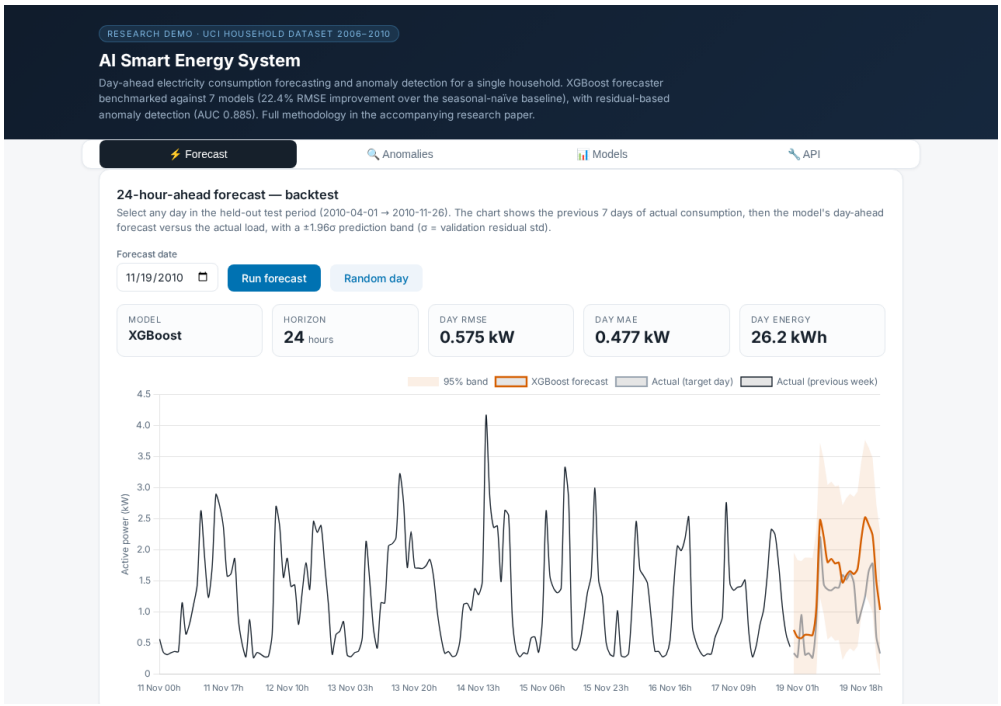


Figure 9: The deployed dashboard (local instance): day-ahead forecast backtest with prediction band, day-level metrics, and navigation to the anomaly, leaderboard, and API views.

code, trained models, figures, the research paper, a Kaggle training notebook, and a one-click Render deployment, so that both the results and the running system can be independently verified and extended.

References

[1] Hesham K. Alfares and Mohammad Nazeeruddin. Electric load forecasting: Literature survey and classification of methods. *International Journal of Systems Science*, 33(1):23–34, 2002.

[2] Tao Hong and Shu Fan. Probabilistic electric load forecasting: A tutorial review. *International Journal of Forecasting*, 32(3):914–938, 2016.

[3] Wei Kong, Zhao Yang Dong, Youwei Jia, David J. Hill, Yan Xu, and Yuan Zhang. Short-term residential load forecasting based on LSTM recurrent neural networks. *IEEE Transactions on Smart Grid*, 10(1):841–851, 2019.

[4] Peter R. Winters. Forecasting sales by exponentially weighted moving averages. *Management Science*, 6(3):324–342, 1960.

[5] Robert B. Cleveland, William S. Cleveland, Jean E. McRae, and Irma Terpenning. STL: A seasonal-trend decomposition procedure based on loess. *Journal of Official Statistics*, 6(1):3–73, 1990.

[6] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

[7] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 785–794, 2016.

[8] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems 30 (NeurIPS)*, pages 3146–3154, 2017.

[9] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

[10] Varun Chandola, Arindam Banerjee, and Vipin Kumar. Anomaly detection: A survey. *ACM Computing Surveys*, 41(3): 15:1–15:58, 2009.

[11] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation forest. In *2008 Eighth IEEE International Conference on Data Mining (ICDM)*, pages 413–422, 2008.

[12] Bernhard Schölkopf, John C. Platt, John Shawe-Taylor, Alex J. Smola, and Robert C. Williamson. Estimating the support of a high-dimensional distribution. *Neural Computation*, 13(7):1443–1471, 2001.

[13] Mayu Sakurada and Takehisa Yairi. Anomaly detection using autoencoders with nonlinear dimensionality reduction. In *Proceedings of the MLSDA 2014 2nd Workshop on Machine Learning for Sensory Data Analysis (MLSDA)*, pages 4–11, 2014.

[14] Rob J. Hyndman and George Athanasopoulos. *Forecasting: Principles and Practice*. OTexts, Melbourne, Australia, 3rd edition, 2021.

[15] Georges Hebrail and Antoine Berard. Individual household electric power consumption. UCI Machine Learning Repository, 2012. URL https://archive.ics.uci.edu/dataset/235/individual+household+electric+power+consumption.